

Software Design Document

Smart Expense Tracker with Spending Analysis



Submitted by
Asfia Faheem (1078-2M)

Submitted to
Mr. Zaman Ali

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Version 1.0 | 2026

Summary

This Software Design Document (SDD) describes the architectural design, data model, user interface layout, and system behavior of the Smart Expense Tracker with Spending Analysis — a web-based financial management application developed as a Final Year Project.

The system is built using a three-tier architecture following the Model-View-Controller (MVC) design pattern. The frontend is developed using HTML, CSS, JavaScript, and Bootstrap, while the backend is powered by PHP and the database layer uses MySQL.

Key features include user registration and authentication, income and expense recording, category management, monthly budget setting with alerts, a dashboard displaying financial summaries, and visual spending analysis through charts and reports.

This document serves as a technical blueprint for developers and evaluators, covering the application architecture, controller functions, data model schema with an ER diagram, class diagrams, sequence diagrams, and user interface wireframes with design conventions.

Table of Contents

1. Application Architecture.....	1
1.1 MVC Components.....	1
1.1.1 Models.....	1
1.1.2 Views.....	2
1.1.3 Controllers.....	2
1.2 Class Diagram.....	4
1.3 Sequence Diagrams.....	6
1.4 Coding Conventions.....	8
2. Data Model Schema.....	9
2.1 Entity: users.....	9
2.2 Entity: transactions.....	10
2.3 Entity: categories.....	10
2.4 Entity: budgets.....	11
2.5 Entity Relationships.....	12
3. User Interface.....	13
3.1 Design Philosophy.....	13
3.2 List of Screens.....	16
3.3 Login Screen.....	17
3.4 Dashboard Screen.....	19
3.5 Add Expense Screen.....	21
3.6 Add Income Screen.....	23
3.7 Other Screens – Controls Summary.....	25
4. References.....	26

1. Application Architecture

The Smart Expense Tracker is designed using the Model-View-Controller (MVC) architectural pattern. This pattern separates the application into three interconnected components, ensuring clean code organization, maintainability, and scalability.

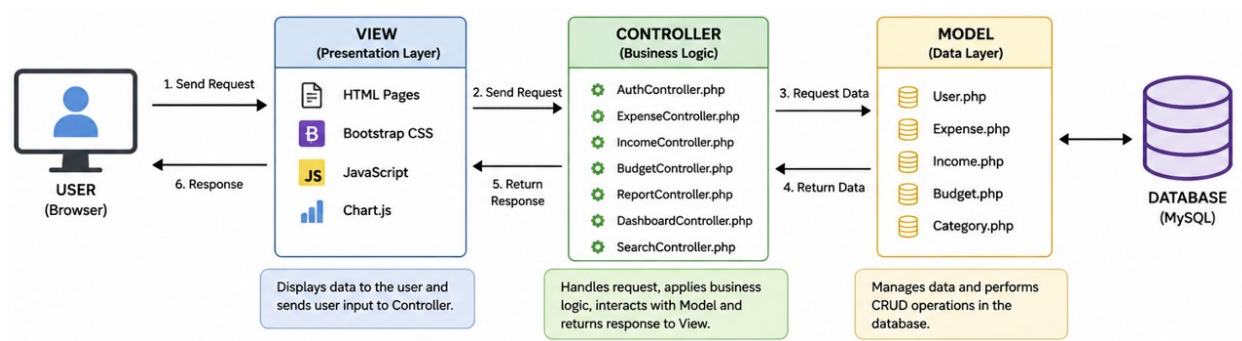


Figure 1 – MVC Application Architecture

1.1 MVC Components

1.1.1 Models

The Model layer is responsible for all data access, business logic, and interaction with the MySQL database. The following models are proposed:

- User – Manages user account data and authentication logic.
- Transaction – Handles income and expense records.
- Budget – Stores and evaluates monthly budget limits.
- Category – Maintains spending categories.
- Report – Generates and retrieves report data.

1.1.2 Views

The View layer presents data to the user through the web browser. The following views are proposed:

- LoginView – Displays the login and registration forms.
- DashboardView – Shows summary cards and chart widgets.
- ExpenseView – Form interface for adding/editing transactions.
- ReportView – Renders spending analysis charts and tabular reports.
- BudgetView – Budget configuration screen.

1.1.3 Controllers

Controllers handle HTTP requests, invoke Model methods, and select the appropriate View to render. The controllers and their key functions are listed below:

Controller	Method / Function	Description
AuthController	register()	Validates input and creates user account
	login()	Authenticates user credentials and starts session
	logout()	Destroys session and redirects to login page
ExpenseController	addExpense()	Validates and saves a new expense record
	addIncome()	Validates and saves a new income record
	editTransaction()	Updates an existing transaction record
	deleteTransaction()	Removes a transaction from the database
	searchFilter()	Filters transactions by date, category, or type
BudgetController	setBudget()	Saves the monthly budget for a user
	checkBudgetAlert()	Compares expenses against budget and triggers alerts
	getBudgetStatus()	Returns current spending versus budget

Controller	Method / Function	Description
ReportController	generateReport()	Compiles transaction data for a selected period
	getChartData()	Returns data formatted for chart rendering
DashboardController	loadDashboard()	Aggregates income, expense, and balance totals
	refreshData()	Updates dashboard data on user action

1.2 Class Diagram

The following class diagram illustrates the relationships between the core entities and their attributes and methods within the system.

What: Represents the static structure of the system including classes, attributes, methods, and relationships. Why: Helps define data ownership, relationships between entities, and supports object-oriented implementation. How: Use UML to show classes, their attributes, methods, and relationships like inheritance, composition, aggregation, and association. Standard Way: Draw clear UML class diagrams with proper visibility symbols (+ public, - private, # protected), relationship notations, and ensure all main classes of the chatbot are included

Symbol / Notation	Name	Meaning
+	Public	Accessible from outside the class
-	Private	Accessible only within the class
#	Protected	Accessible within the class and its subclasses
()	Method	Represents a function or behavior
:	Data Type	Specifies the type of an attribute
→	Association	Shows interaction between classes
▷	Inheritance	Child class inherits from parent class
◆	Composition	Strong ownership (dependent class)
1	One	Single object
*	Many	Multiple objects
1..*	One to Many	One object linked to many objects

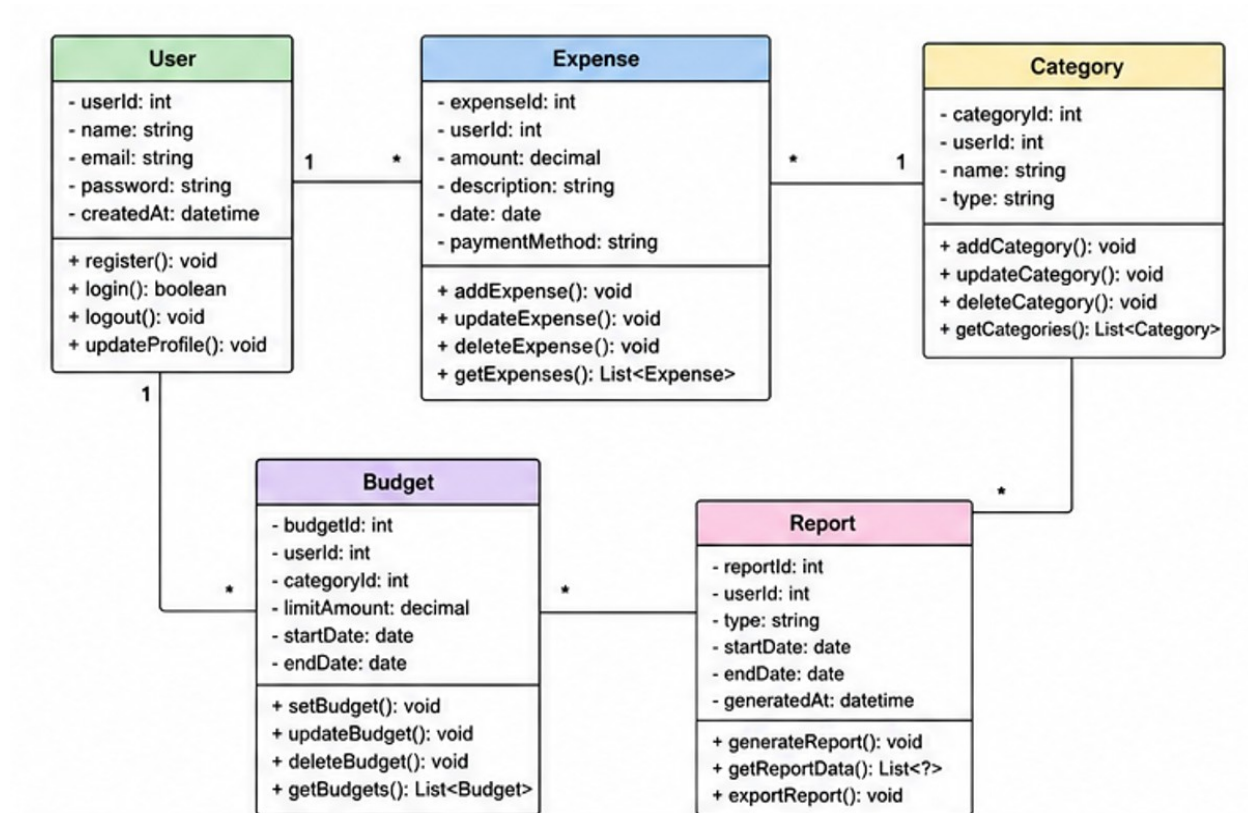


Figure 2 – Class Diagram

1.3 Sequence Diagrams

Sequence diagrams illustrate the interaction between system components for key use cases over time.

What: Shows the interaction between system components over time. Why: Helps explain runtime behavior, message passing, and validates use cases. How: Use actors and lifelines to show objects, messages, and activations in order. Standard Way: Draw UML sequence diagrams with proper lifelines, horizontal arrows for messages, and activation bars to indicate active periods of objects during interactions

Symbol	Name	Purpose
Rectangle (top)	Actor / Object	Represents participants involved in the interaction
Dashed vertical line	Lifeline	Shows the existence of an object over time
Solid horizontal arrow	Message	Represents a method call or request between objects
Dashed horizontal arrow	Return Message	Shows the response or result of a message
Thin vertical rectangle	Activation Bar	Indicates the time an object is actively processing
Loop frame	Combined Fragment (Loop)	Represents repeated interactions during the session

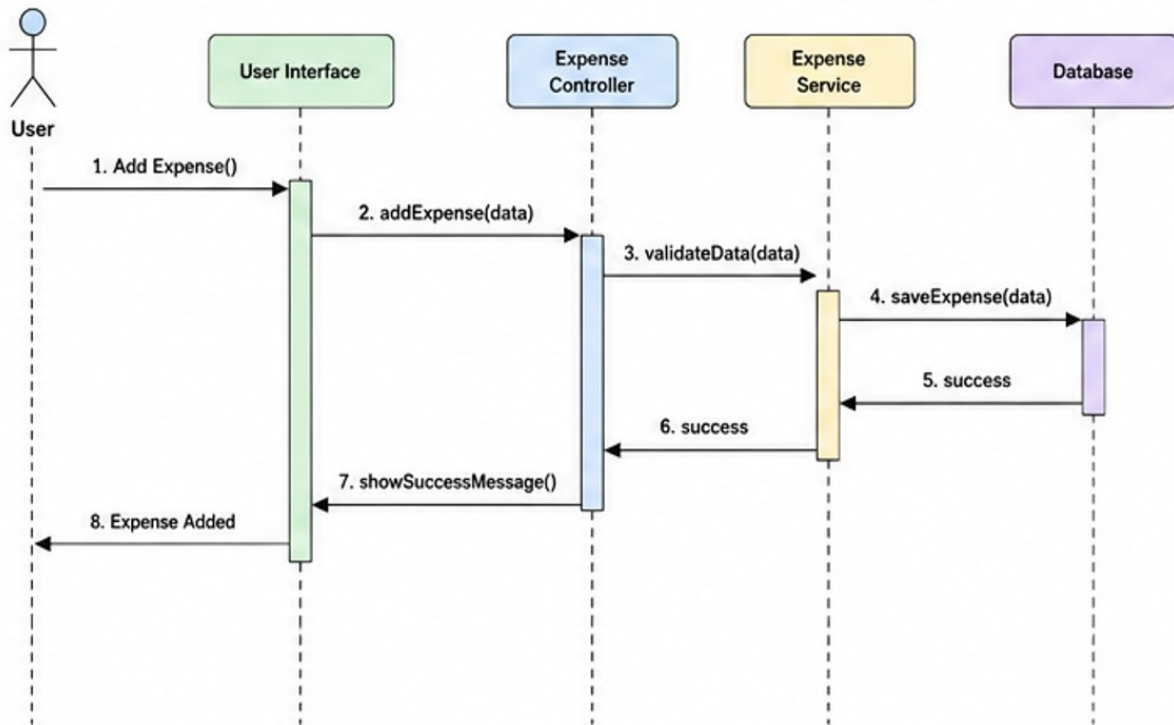


Figure 3– Sequence Diagram

1.4 Coding Conventions

The following coding conventions are adopted throughout the development of this application:

Convention	Standard Applied
Naming – Variables	camelCase (e.g., totalExpense, userID)
Naming – Functions	camelCase verbs (e.g., addExpense(), setBudget())
Naming – Classes	PascalCase (e.g., ExpenseController, UserModel)
Naming – Database	Snake case (e.g., transaction_id, user_email)
Indentation	4 spaces per level (no tabs)
Comments	PHPDoc blocks for functions; inline comments for logic
File Structure	Separate folders: /models, /views, /controllers, /assets
SQL	Prepared statements for all queries to prevent SQL injection

2. Data Model Schema

This section describes the conceptual database schema of the Smart Expense Tracker. The system uses a relational MySQL database. Each entity, its properties, and its relationships with other entities are explained below.

2.1 Entity: users

Stores registered user accounts and authentication credentials.

Column	Data Type	Constraint	Description
user_id	INT	PK, AUTO_INCREMENT	Unique identifier for each user
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(150)	UNIQUE, NOT NULL	Email address used for login
password_hash	VARCHAR(255)	NOT NULL	Hashed password (bcrypt)
Created_at	DATETIME	DEFAULT NOW()	Account creation timestamp

2.2 Entity: transactions

Stores all income and expense records entered by users.

Column	Data Type	Constraint	Description
transaction_id	INT	PK, AUTO_INCREMENT	Unique transaction identifier
user_id	INT	FK → users	Reference to the owning user
amount	DECIMAL(10,2)	NOT NULL	Monetary value of the transaction
type	ENUM('income','expense')	NOT NULL	Indicates if the record is income or expense
category_id	INT	FK → categories	Category to which the transaction belongs
transaction_date	DATE	NOT NULL	Date the transaction occurred
notes	TEXT	NULLABLE	Optional description or note
created_at	DATETIME	DEFAULT NOW()	Record creation timestamp

2.3 Entity: categories

Stores predefined and user-defined spending/income categories.

Column	Data Type	Constraint	Description
category_id	INT	PK, AUTO_INCREMENT	Unique category identifier
name	VARCHAR(80)	NOT NULL	Category name (e.g., Food, Transport)
type	ENUM('income','expense')	NOT NULL	Whether this category is for income or expense

2.4 Entity: budgets

Stores monthly budget limits set by each user.

Column	Data Type	Constraint	Description
budget_id	INT	PK, AUTO_INCREMENT	Unique budget record identifier
user_id	INT	FK → users	Reference to the owning user
month_year	VARCHAR(7)	NOT NULL	Budget period in MM-YYYY format
monthly_limit	DECIMAL(10,2)	NOT NULL	Maximum spending limit for the month
alert_threshold	DECIMAL(5,2)	DEFAULT 80.00	Percentage at which alert is triggered

2.5 Entity Relationships

- A User can have many Transactions (one-to-many).

- A User can have many Budgets — one per month (one-to-many).
- A Transaction belongs to one Category (many-to-one).
- A Category can be linked to many Transactions (one-to-many).

What: Represents the system’s data entities, their attributes, and the relationships between them. Why: Ensures clear, normalized, and structured data storage, avoiding redundancy and maintaining data integrity. How: Identify entities, define primary keys, attributes, and relationships (1:1, 1:N, N:M), and normalize the schema.

Standard Way: Draw ERD using rectangles for entities, ovals for attributes, diamonds for relationships, and connect entities with lines labeled with cardinality; ensure 1NF, 2NF, and 3NF are applied for all entities. “The ERD was designed by identifying real-world entities involved in the system and defining their attributes and relationships.

Primary and foreign keys were used to ensure data integrity, and normalization rules were applied to remove redundancy. The diagram reflects real system behavior and supports secure, structured, and scalable database design.” Notation Rectangle PK Meaning Entity Primary Key (unique identifier)

The following ER diagram visually represents these relationships:

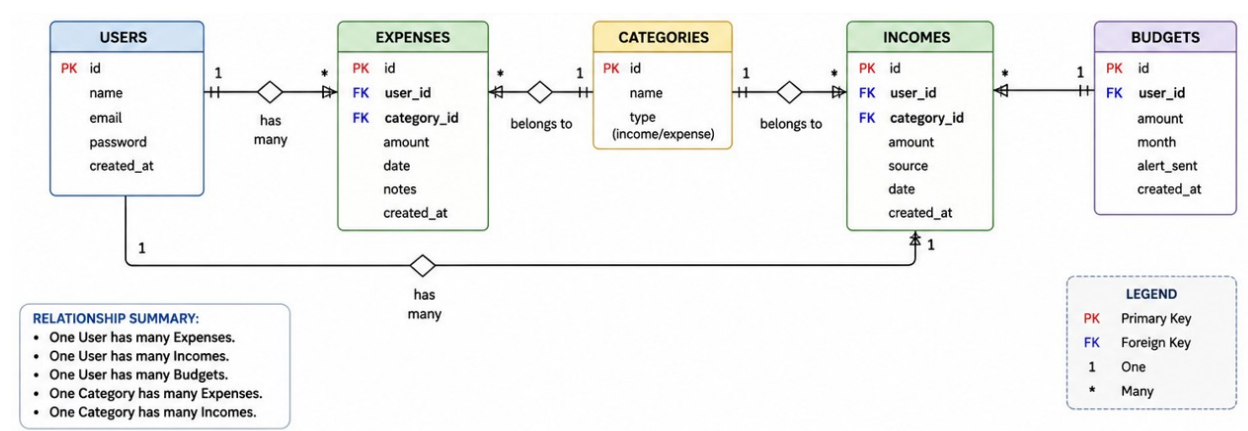


Figure 4– Entity-Relationship (ER) Diagram

3. User Interface

3.1 Design Philosophy

The **Smart Expense Tracker** is designed with a strong focus on **simplicity, usability, accessibility, and efficiency**. The main objective of the application is to help users manage their personal finances in an organized manner without requiring advanced technical knowledge. The interface is kept clean and user-friendly so that users can easily record income, track expenses, set budgets, and analyze spending patterns.

Color Scheme

The application follows a simple and professional color scheme to improve readability and user experience. A **white background** is used throughout the system to create a clean appearance, while **dark-colored text** ensures better visibility. Different colors are used to represent different financial activities. **Green** is used for income-related information, positive balances, and success messages, whereas **red** is used for expenses, warning messages, and over-budget alerts. This color distinction helps users quickly identify important financial information.

Typography

Typography plays an important role in maintaining readability and consistency. The application uses the **Roboto (sans-serif)** font family because of its modern and professional appearance. Headings are displayed in **bold fonts ranging from 20px to 24px** to create a clear visual hierarchy. Body text is maintained between **14px and 16px** to ensure comfortable reading across different screen sizes. Consistent font styles help users understand and navigate the interface more effectively.

Design Framework

The user interface is developed using **Bootstrap 5**, a widely used front-end framework. Bootstrap provides a responsive grid system, pre-built components, and utility classes that simplify interface

development. Using Bootstrap ensures consistency across all pages and improves responsiveness on different devices.

Screen Targets

The application follows a **desktop-first design approach**, primarily optimized for screens with a width of **1280 pixels or larger**. However, the interface remains responsive and functional on smaller devices, including tablets with screen widths of approximately **768 pixels**. This ensures a smooth user experience across multiple screen sizes.

Layout

The overall layout of the application consists of a **fixed sidebar navigation menu** on the left side and a **main content area** on the right side. The sidebar provides quick access to important modules such as **Dashboard, Add Expense, Add Income, Records, Reports, Budget, and Logout**. This layout improves navigation efficiency and allows users to move between different sections without confusion.

Charts and Data Visualization

To help users better understand their financial activities, the system uses the **Chart.js** library for data visualization. Interactive **pie charts** are used to display expense distribution across categories, while **bar charts** are used to show monthly spending trends and comparisons. These visual representations make financial analysis easier and more understandable than viewing raw data alone.

Accessibility

Accessibility is an important consideration in the design of the Smart Expense Tracker. All form elements contain **descriptive labels** to improve usability. The interface maintains **sufficient color**

contrast according to WCAG AA standards, ensuring readability for a wide range of users. Error messages are clearly displayed, and navigation elements are designed to be intuitive. These features make the application accessible and easy to use for individuals with basic computer literacy.

Overall Design Goal

The overall goal of the design is to provide a **clean, responsive, and user-friendly financial management system** that enables users to efficiently track their income and expenses, monitor budgets, and gain meaningful insights into their spending behavior through an intuitive and visually appealing interface.

3.2 List of Screens

#	Screen Name	Purpose
1	Registration	Allows new users to create an account
2	Login	Authenticates returning users
3	Dashboard	Displays financial summary, balance, and quick charts
4	Add Expense	Form to record a new expense
5	Add Income	Form to record a new income entry
6	Transaction List	Paginated list of all income/expense records with search and filter
7	Edit / Delete	Inline edit form and delete confirmation for a selected record
8	Budget Setting	Form to set monthly budget limit and alert threshold
9	Spending Analysis	Visual reports — pie chart by category, bar chart by month
10	Profile / Settings	User profile information and password change

3.3 Login Screen

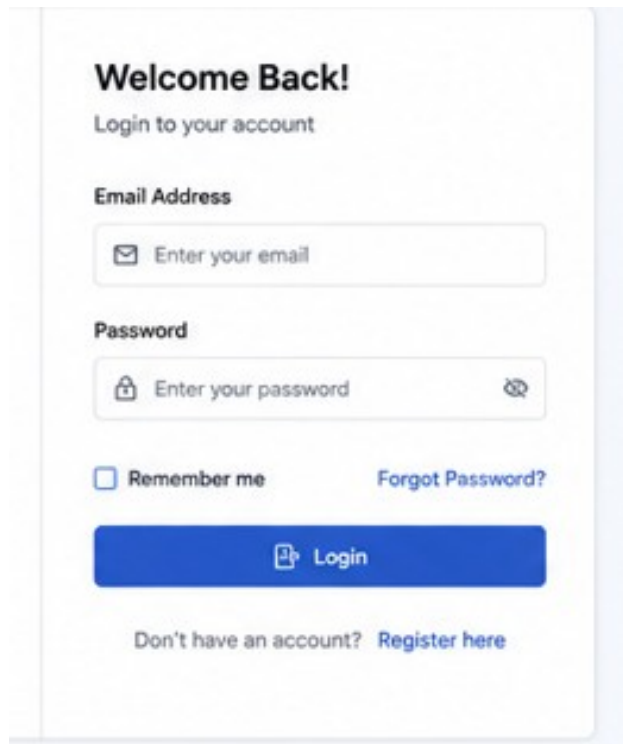
This screen allows registered users to authenticate and access the application. It is the entry point of the Smart Expense Tracker for returning users.

Data on Screen:

- Email Address – Text input field for the registered user email.
- Password – Masked input field for the user password.
- Forgot Password Link – Navigates to the password reset flow.
- Register Link – Redirects new users to the Registration screen.

Controls:

Control	Type	Action
Email	Text Input	Accepts registered email address; validated on submit
Password	Password Input	Masked input; validated against stored hash on submit
Login	Button (Primary)	Submits credentials; starts session on success and redirects to Dashboard
Forgot Password	Hyperlink	Navigates to password reset page



The image shows a login screen with a white background and a light blue border. At the top, it says "Welcome Back!" in bold, followed by "Login to your account". Below this, there are two input fields: "Email Address" with a placeholder "Enter your email" and an envelope icon, and "Password" with a placeholder "Enter your password", a lock icon, and a toggle icon. Under the password field, there is a checkbox labeled "Remember me" and a link "Forgot Password?". A large blue button with a white login icon and the text "Login" is positioned below these fields. At the bottom, there is a link "Don't have an account? Register here".

Figure 5 – login screen

3.4 Dashboard Screen

The dashboard is the primary screen the user sees after login. It presents an at-a-glance summary of the financial state.

Data Displayed:

- Total Income – Aggregated income for the current month.
- Total Expenses – Aggregated expenses for the current month.
- Remaining Balance – Calculated as Total Income minus Total Expenses.
- Budget Left – Remaining budget before alert threshold is reached.
- Spending by Category – Pie chart rendered by Chart.js.
- Monthly Expense Trend – Bar chart showing last 6 months of expenses.
- Recent Transactions – Last 5 transactions in a summary row.

Controls:

- Navigation Bar – Links to Dashboard, Add Expense, Reports, and Logout.
- Quick Action Buttons – Add Expense, Add Income.

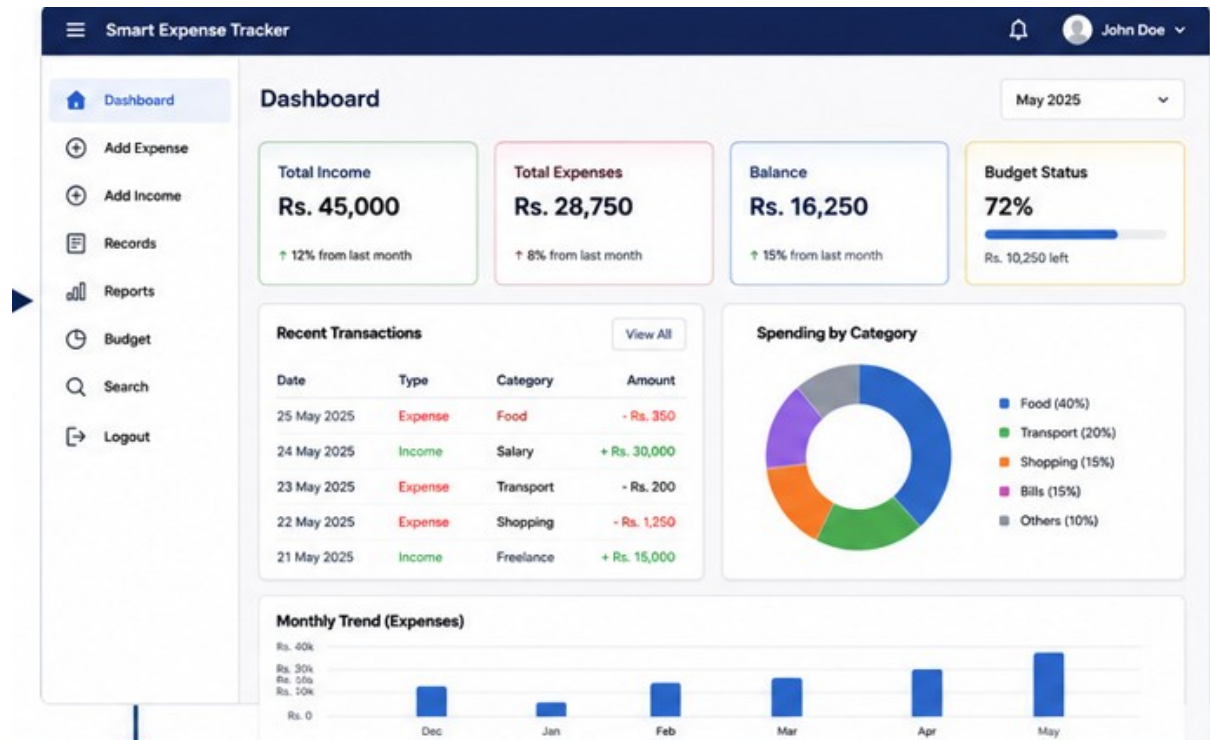


Figure 6– Dashboard Screen Wireframe

3.5 Add Expense Screen

This screen provides a form for the user to enter a new expense record.

Data on Screen:

- Amount – Numeric input field for the expense value in PKR.
- Category – Dropdown populated from the categories table.
- Date – Date picker defaulting to today.
- Notes – Optional textarea for additional description.

Controls:

Control	Type	Action
Amount	Number Input	Accepts numeric values only; validated on submit
Category	Dropdown (Select)	Lists all expense categories from the database
Date	Date Picker	Defaults to current date; user can change
Notes	Textarea	Optional; maximum 300 characters
Save Expense	Button (Primary)	Submits the form; triggers validation and DB insert
Cancel	Button (Secondary)	Discards input and returns to the transaction list

Smart Expense Tracker

Dashboard > Add Expense

Add Expense

Expense Information

Amount *
Enter amount PKR

Category *
Select category

Date *
dd/mm/yyyy

Notes (Optional)
Enter notes (optional)

Cancel **Save Expense**

Add New Expense
Fill in the details of your expense.
All fields marked with * are required.

Tips

- Select the correct category.
- Enter accurate amount.
- Add notes for better tracking.

Figure 7 – Add expense screen

3.6 Add Income Screen

This screen provides a form for the user to record a new income entry. It mirrors the Add Expense screen in layout but is configured for income-type transactions.

Data on Screen:

- Amount – Numeric input field for the income value in PKR.
- Category – Dropdown populated with income categories from the database.
- Date – Date picker defaulting to today's date.
- Notes – Optional textarea for additional description of the income source.

Controls:

Control	Type	Action
Amount	Number Input	Accepts numeric values only; validated on submit
Category	Dropdown (Select)	Lists all income categories from the database
Date	Date Picker	Defaults to current date; user can change
Notes	Textarea	Optional; maximum 300 characters
Save Income	Button (Primary)	Submits the form; triggers validation and DB insert for income record
Cancel	Button (Secondary)	Discards input and returns to the transaction list

The screenshot displays the 'Add Income' form within the 'Smart Expense Tracker' application. The interface includes a dark blue header with the application name, a notification bell, and a user profile for 'John Doe'. A left sidebar contains navigation links: Dashboard, Add Expense, Add Income (highlighted), Records, Reports, Budget, Search, and Logout. The main content area is titled 'Add Income' and shows a breadcrumb 'Dashboard > Add Income'. The form is divided into two main sections: 'Income Information' and 'Add New Income'.

Income Information

Amount *

Enter amount PKR

Source *

Enter source (e.g., Salary, Freelance)

Category *

Select category

Date *

dd/mm/yyyy

Notes (Optional)

Enter notes (optional)

Buttons: Cancel, Save Income

Add New Income

Fill in the details of your income.
All fields marked with * are required.

Tips

- Select the correct category.
- Enter accurate amount.
- Add notes for better tracking.

Figure 4 – Add income

3.7 Other Screens – Controls Summary

Screen	Key Controls
Registration	Textboxes: Name, Email, Password, Confirm Password; Register Button
Login	Textboxes: Email, Password; Login Button; Forgot Password Link
Transaction List	Search Textbox, Date Range Picker, Category Filter Dropdown, Type Filter (Income/Expense), Paginator
Budget Setting	Month-Year Picker, Amount Input, Alert Threshold Slider (%), Save Budget Button
Spending Analysis	Period Selector (Month/Year), Pie Chart (by category), Bar Chart (monthly trend), Export to PDF Button
Profile / Settings	Name Textbox, Email Textbox (read-only), Change Password Form, Save Button

4. References

[1] NerdWallet, "Personal Finance and Expense Tracking," [Online]. Available: <https://www.nerdwallet.com/>, Accessed: Apr. 28, 2026.

[2] TechRadar, "Technology News and Reviews," [Online]. Available: <https://www.techradar.com/>, Accessed: Apr. 28, 2026.

[3] Intuit, "Accounting Software for Businesses," [Online]. Available: <https://quickbooks.intuit.com/>, Accessed: Apr. 28, 2026.

[4] Bootstrap, "Bootstrap 5 Documentation," [Online]. Available: <https://getbootstrap.com/docs/5.0/>, Accessed: May 2026.

[5] Chart.js, "Chart.js Open Source HTML5 Charts," [Online]. Available: <https://www.chartjs.org/>, Accessed: May 2026.

[6] PHP Manual, "PHP: Hypertext Preprocessor," [Online]. Available: <https://www.php.net/manual/>, Accessed: May 2026.

[7] MySQL, "MySQL 8.0 Reference Manual," [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>, Accessed: May 2026.